Rapport de stage BTS SIO 2

Du 05/01/2026 au 20/02/2026

Stage de développement full-stack au sein d'Ateja

Yasmine Henni-Zourgui

INTRODUCTION GÉNÉRALE

Dans le cadre de ma deuxième année de BTS Services Informatiques aux Organisations, option SLAM, j'ai réalisé un stage de sept semaines au sein de l'entreprise Ateja.

L'objectif de ce stage était de m'intégrer à une équipe de développement et de participer à l'évolution d'une application déjà utilisée en production. J'ai ainsi pu mettre en pratique les compétences techniques acquises durant ma formation, tout en développant de nouvelles connaissances liées au développement d'applications web. Ce stage m'a également permis de mieux comprendre l'organisation d'un service informatique, les contraintes liées à la maintenance d'un projet existant et le niveau d'exigence attendu en termes de qualité, de rigueur et de travail en équipe.

J'ai donc effectué mon stage chez Ateja, une entreprise basée à Villeneuve d'Ascq, chargée de la maintenance d'un ERP : Eden. Un ERP, acronyme de *Enterprise Resource Planning*, est un système d'information centralisé permettant de gérer l'ensemble des processus opérationnels d'une organisation au sein d'une base de données unique.

Un ERP regroupe généralement plusieurs modules fonctionnels tels que la gestion commerciale, la comptabilité, la gestion des ressources humaines, la gestion des stocks ou encore le suivi des projets. L'objectif principal d'un tel système est d'assurer la cohérence et l'unification des données, tout en facilitant la circulation de l'information entre les différents services d'une entreprise.

Dans une architecture ERP, les différents modules partagent une base de données commune. Cette centralisation permet d'éviter les redondances d'informations, de limiter les erreurs de saisie et d'améliorer la traçabilité des opérations. Par exemple, une modification effectuée dans un module peut automatiquement avoir un impact sur un autre module, garantissant ainsi la cohérence globale du système. L'application Eden s'inscrit dans cette logique d'intégration et de centralisation des données.

Mon travail s'est concentré sur trois situations professionnelles : l'automatisation de la gestion des dépendances du projet, l'amélioration du suivi des migrations de base de données et la conception d'un système de validation conditionnelle des champs dans les formulaires.

Table des matières

INTRODUCTION GÉNÉRALE *1*

1. PRÉSENTATION DE L'ENTREPRISE *3*

1.1 Activité et domaine d'intervention **3**

1.2 Organisation du service informatique **3**

1.3 Environnement technique et architecture logicielle **4**

1.4 Gestion du patrimoine informatique et veille technologique **5**

2. SITUATION PROFESSIONNELLE N°1 ..... *5*

Automatisation de la gestion des dépendances Composer 5

2.1 Contexte et problématique **5**

2.2 Analyse technique du besoin **5**

2.3 Conception et mise en œuvre **6**

2.4 Difficultés rencontrées et solutions apportées **7**

2.5 Apports professionnels **7**

3. SITUATION PROFESSIONNELLE N°2 ..... *8*

Amélioration du suivi et de l'affichage des migrations 8

3.1 Contexte général **8**

3.2 Analyse fonctionnelle et technique du besoin **8**

3.3 Conception de la solution **8**

3.4 Tests et validation **11**

3.5 Difficultés rencontrées **11**

3.6 Apports professionnels **11**

4. SITUATION PROFESSIONNELLE N°3 ..... *12*

Conception et implémentation d'un système de validation conditionnelle des champs 12

4.1 Contexte métier **12**

4.2 Analyse fonctionnelle approfondie **12**

4.3 Conception de la base de données **13**

4.4 Implémentation côté serveur **13**

4.5 Implémentation côté client **16**

4.6 Difficultés rencontrées **17**

4.7 Apports professionnels **18**

5. BILAN PROFESSIONNEL ET PERSONNEL *18*

1. PRÉSENTATION DE L'ENTREPRISE

1.1 Activité et domaine d'intervention

Ateja a pour activité principale le développement d'un ERP, Eden, accessible en ligne, permettant à ses clients de gérer différents processus métiers via une interface web sécurisée. Eden est conçu de manière modulaire, ce qui permet d'adapter la solution aux besoins spécifiques de chaque client.

Cette organisation en modules signifie que le logiciel est conçu pour évoluer facilement. Il est possible d'ajouter de nouvelles fonctionnalités régulièrement sans perturber le bon fonctionnement de celles qui existent déjà.

L'entreprise ne se limite pas au développement initial de l'application. Elle assure également la maintenance corrective, la correction des erreurs signalées par les utilisateurs. En parallèle, elle réalise une maintenance évolutive en proposant des améliorations et des mises à jour régulières afin d'adapter l'application aux nouveaux besoins

Les clients de l'entreprise sont des structures professionnelles de taille moyenne ayant besoin d'un outil de gestion personnalisé. Les partenaires techniques incluent notamment des hébergeurs assurant la mise en ligne des applications, ainsi que les communautés open source fournissant les frameworks et outils utilisés.

1.2 Organisation du service informatique

J'ai été intégré au sein du service développement, composé de 3 développeurs full-stack, un développeur spécialisé en infrastructure et un chef de projet.
Dans l'équipe d'Ateja on retrouve également les intégrateurs qui travaillent avec les clients directement.

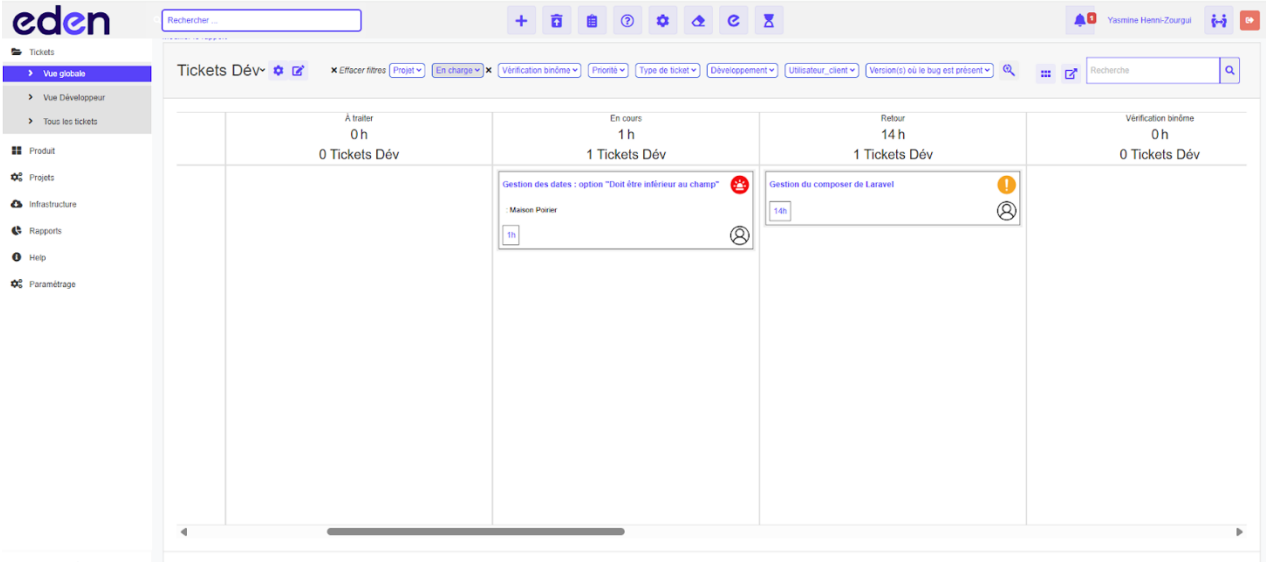
L'organisation du travail repose sur une méthodologie Agile, favorisant la collaboration. Des réunions journalières (le daily scrum) sont organisées afin de présenter les avancées, d'identifier les difficultés éventuelles et de valider les choix techniques de chaque membre de l'équipe. Cette démarche favorise la communication interne et permet de garantir la cohérence globale de l'application.

Toute les 3 semaines, la démonstration, avec l'ensemble de l'équipe d'Ateja permet de présenter toutes les nouveautés de la nouvelle version de l'application avant de commencer à l'implémenter.

Après la démonstration, les développeurs font entre eux la rétrospective des 3 semaines passées afin d'exprimer leurs réussites, leurs difficultés, leur retour global sur le sprint précédent.

Chaque demande d'évolution ou de correction est formalisée sous forme de ticket détaillant le contexte, le besoin et les contraintes techniques éventuelles. La gestion des tickets se fait sur la plateforme interne d'Eden.

Ces tickets sont analysés puis attribués en fonction de la charge de travail et des compétences requises. Cette organisation permet de structurer les développements tout en maintenant une certaine souplesse dans la gestion des priorités.



Capture d'écran de mes tickets

1.3 Environnement technique et architecture logicielle

L'application Eden repose sur le framework Laravel pour la partie back-end. Laravel est un framework PHP structuré selon le modèle MVC (Modèle-Vue-Contrôleur), ce qui permet de séparer clairement la gestion des données, la logique métier et l'affichage. Cette architecture favorise la maintenabilité et l'évolution du code.

La base de données utilisée est une base relationnelle, ici MariaDB, administrée à l'aide de phpMyAdmin. Elle stocke l'ensemble des données nécessaires au fonctionnement de l'application.

L'application communique avec cette base de données grâce à l'ORM Eloquent, intégré au framework Laravel. Cet outil permet de manipuler les données sous forme d'objets plutôt que d'écrire directement des requêtes SQL. Cela simplifie les opérations courantes, comme l'insertion, la modification ou la suppression d'enregistrements, tout en conservant la possibilité d'utiliser des requêtes SQL plus complexes lorsque cela est nécessaire.

La partie front-end utilise Vue.js, un framework JavaScript permettant de créer des interfaces dynamiques et interactives. Les composants Vue.js permettent d'isoler certaines fonctionnalités de l'interface et de rendre l'affichage réactif aux modifications de données. Les échanges entre le front-end et le back-end sont réalisés par des requêtes AJAX, assurant une communication asynchrone sans rechargement complet de la page.

La gestion des dépendances est assurée par Composer, tandis que le versionnement du code est réalisé via Git et hébergé sur Github. Cette organisation garantit la traçabilité des modifications et la stabilité des versions mises en production.

1.4 Gestion du patrimoine informatique et veille technologique

La gestion du patrimoine informatique repose sur une organisation rigoureuse des ressources matérielles et logicielles. Les postes de développement sont configurés afin de reproduire au plus près l'environnement de production. Chaque évolution est testée localement avant d'être intégrée dans la branche principale.

La veille technologique est assurée principalement grâce à l'outil Feedly. Chaque semaine, un membre différent de l'équipe est chargé de cette mission, ce qui permet d'impliquer l'ensemble des développeurs et de partager régulièrement les nouvelles informations avec le reste du service.

Feedly est un agrégateur de flux RSS. Il s'agit d'une plateforme en ligne qui permet de regrouper et de centraliser les publications issues de différents sites spécialisés, blogs techniques et médias numériques. Grâce à cet outil, l'équipe peut suivre l'actualité des technologies utilisées, comme Laravel, Vue.js ou encore les bonnes pratiques en développement web, sans avoir à consulter manuellement chaque site.

2. SITUATION PROFESSIONNELLE N°1

Automatisation de la gestion des dépendances Composer

2.1 Contexte et problématique

Lors de la création d'un nouveau projet client, l'entreprise procédait à la duplication d'un projet existant. L'application comprend un module commun partagé entre plusieurs projets, contenant des dépendances communes. Lorsque de nouvelles dépendances étaient ajoutées à ce module, il était nécessaire de modifier manuellement le fichier composer.json du projet principal afin d'y intégrer ces ajouts.

Cette méthode présentait plusieurs inconvénients. Elle nécessitait une intervention répétitive, augmentait le risque d'erreur humaine et rendait la maintenance plus complexe. Une omission dans la mise à jour du fichier pouvait entraîner des dysfonctionnements ou des incompatibilités.
Il était donc nécessaire de mettre en place une solution automatisée permettant de garantir la cohérence des dépendances entre les différents projets.

2.2 Analyse technique du besoin

Composer est un gestionnaire de dépendances utilisé dans les projets PHP. Il permet de déclarer les bibliothèques nécessaires au bon fonctionnement d'une application à travers un fichier `composer.json`, dans lequel sont listés les packages requis ainsi que leurs versions.

Dans le contexte d'Eden, l'architecture des projets reposait sur deux fichiers distincts. Le premier contenait les dépendances communes à l'ensemble de l'application Eden, tandis que le second regroupait les dépendances spécifiques à un projet client particulier.

Le besoin consistait à fusionner automatiquement ces deux fichiers de dépendances afin de générer un fichier **`composer.json`** unique à la racine du projet. Une fois cette fusion réalisée, la commande **`composer update`** devait être exécutée automatiquement afin d'installer ou de mettre à jour les packages nécessaires. L'objectif était d'éviter toute modification manuelle du fichier principal et de simplifier la maintenance des projets, en intégrant directement cette automatisation dans l'application pour qu'elle soit facilement accessible aux développeurs.

2.3 Conception et mise en œuvre

Pour répondre à ce besoin, j'ai développé une méthode statique permettant de lire le contenu du fichier composer.json du module commun ainsi que celui qui est spécifique au projet. Les deux fichiers sont fusionnés à l'aide d'une fonction adaptée, garantissant que les dépendances communes et spécifiques soient toutes présentes dans le fichier final.
Le fichier généré est ensuite écrit à la racine du projet. Une fois cette étape réalisée, la commande « composer update » est exécutée via la classe Process fournie par Laravel. Cette classe permet d'exécuter des commandes système de manière sécurisée depuis l'application.
Une attention particulière a été portée à la gestion des erreurs. En cas de problème d'écriture du fichier ou d'échec de la commande système, un message explicite est renvoyé afin d'orienter l'utilisateur vers la résolution du problème.

```php
public static function mise_a_jour_composer() {

    $composerSource = base_path('app/Eden/composer.json');
    $composerSpecifique = base_path('composer_specifique.json');
    $commun = json_decode(file_get_contents($composerSource), true);


    //si le composer spécifique existe on fusionne les require
   if(file_exists($composerSpecifique) ) {
        $specifique = json_decode(file_get_contents($composerSpecifique), true);

        $commun['require'] = array_merge(
        $commun['require'] ?? [],
        $specifique['require'] ?? []
    );
    }
    //création/maj du nouveau composer.json
    $composerCible = base_path('composer.json');
    $retour = file_put_contents($composerCible,json_encode($commun, JSON_PRETTY_PRINT | JSON_UNESCAPED_SLASHES));

    //si retour true on lance le composer update
    if($retour !== false) {
        $result = Process::path(base_path())
            ->timeout(600)
            ->run('composer update');
        $result->output();

        if($result->successful())
            return true;

        return "Erreur exécution composer update\n\n"
        . "Faire les commandes suivantes à la racine du projet puis relancer la mise à jour:\n\n"
        . "sudo chown -R www-data:www-data /var/www/eden/vendor\n"
        . "sudo chmod -R 775 /var/www/eden/vendor\n\n"
        ;

    }

    return 'Erreur création composer.json' ;
}
```

2.4 Difficultés rencontrées et solutions apportées

La principale difficulté rencontrée concernait les droits d'accès au dossier « vendor ». Lors de l'exécution de la commande depuis l'application, il arrivait que l'utilisateur système ne dispose pas des autorisations nécessaires. Après analyse, j'ai intégré un message d'erreur détaillant les commandes permettant de corriger les droits d'accès.

2.5 Apports professionnels

Cette mission m'a permis de gérer le patrimoine informatique en analysant les dépendances nécessaires au bon fonctionnement de l'application et en garantissant leur cohérence entre les projets. L'automatisation mise en place contribue à assurer la continuité du service et à limiter les erreurs humaines.

J'ai également répondu à une demande d'évolution applicative formalisée sous forme de ticket. Le travail a été mené en mode projet : analyse du besoin, conception, développement, tests et validation en équipe.

3. SITUATION PROFESSIONNELLE N°2

Amélioration du suivi et de l'affichage des migrations

3.1 Contexte général

Dans le cadre de la maintenance et de la mise à jour de l'application Eden, les migrations de base de données occupent une place centrale. Une migration correspond à une modification de la base de données, qu'il s'agisse de créer une nouvelle table, d'ajouter une colonne ou de modifier une contrainte existante. Ces migrations sont nécessaires pour faire évoluer la structure des données tout en garantissant la cohérence du système.

L'application disposait déjà d'une interface permettant de lancer ces migrations depuis un écran dédié. Toutefois, l'interface initiale présentait plusieurs limites. Un unique loader affichait le processus en cours, un seul indicateur visuel signalait l'exécution globale du processus, et seule la durée totale était affichée à la fin de l'opération. En cas d'erreur, il était difficile d'identifier précisément quelle migration avait échoué et à quel moment.

Cette situation représentait un frein à la maintenance, car les développeurs perdaient du temps à analyser les logs d'erreur pour identifier l'origine du problème. Il devenait donc nécessaire d'améliorer le suivi et la lisibilité des informations affichées.

3.2 Analyse fonctionnelle et technique du besoin

Le besoin exprimé consistait à rendre l'exécution des migrations plus transparente et plus informative. Il s'agissait notamment d'afficher un loader individuel pour chaque migration, de signaler explicitement son succès ou son échec, et d'indiquer la durée d'exécution de chacune d'elles.

D'un point de vue technique, cela impliquait une modification de la partie front-end et de la partie back-end.
Le front-end devait être capable d'exécuter les migrations de manière séquentielle, d'attendre la réponse du serveur avant de passer à la suivante et d'actualiser dynamiquement l'interface.
Le back-end devait, quant à lui, retourner des réponses structurées au format JSON afin de permettre un traitement clair des informations côté client.
Cette évolution devait également garantir qu'en cas d'erreur sur une migration, le processus s'arrête immédiatement afin d'éviter des incohérences potentielles dans la base de données.

3.3 Conception de la solution

La solution retenue repose sur l'utilisation d'une fonction asynchrone en JavaScript. Cette fonction parcourt un tableau représentant les différentes migrations à exécuter.

```
migrations : [
        {
            name : "gestion_composer",
            enable : true,
            result : null,
            label : "Mise à jour du composer",
            loading: false,
            status : null,
            duree : null,
        },

            {
                name : "gestion_migrations",
                enable : true,
                result : null,
                label : "Mise à jour de la base de données",
                loading: false,
                status : null,
                duree : null,
            },
            {
                name : "gestion_rappels_version",
                enable : true,
                result : null,
                label : "Récupération des rappels de version",
                loading: false,
                status : null,
                duree : null,
            },
            {
                name : "gestion_cache",
                enable : true,
                result : null,
                label : "Nettoyage du cache",
                loading: false,
                status : null,
                duree : null,
            },
    ],
```

Dans la vue, on répertorie les différentes migrations à effectuer, qui si cochée, appelleront une fonction, propre à chacune, du contrôleur.

```
lancer_migrations: async function(event) {


        var instance = this;
        this.bouton_lancement_disabled = true;


        // paramètres communs
        var parametres = {
            url: "{{ route('maintenance.migrations_traitement') }}",
            dataType: "json",
            method: "POST"
        };

        for (value of instance.migrations) {

            if (!value.enable) {
                continue;
            }
            value.loading = true;


            var data = {
                fonction : value.name,
            };

            const debut = performance.now();

            try{
                var retour = await $.ajax({
                    //les ... copie le contenu de parametres dans cet objet
                    ...parametres,
                    data: data
                }).fail(() => {
                    console.log(`La requête pour ${value.name} a échoué.`);
                    this.temps_execution = '<div class="alert alert-danger">Echec des migrations <span style="font-size:20px;">😟</span> !';
                    this.loader_migrations = false;
                    this.bouton_lancement_disabled = false;
                    loading(false);

                    instance.recuperer_rappels_version();
                });
                this.loader_migrations = false;

                const fin = performance.now();
                value.duree = ((fin - debut) / 1000).toFixed(2) + 's';

                value.status = retour.success ? 'success' : 'error';

                if (retour.success) {


                } if(retour.erreur) {
                    instance.erreur_migration = retour.erreur;
                    instance.ligne_erreur_migration = retour.ligne_erreur;
                    instance.fichier_erreur_migration = retour.fichier_erreur;
                    break; // on sort de la boucle en cas d'erreur
                }}
                catch (error) {
                    console.error(`Erreur lors de la requête pour ${value.name}:`, error);
                    break;
            }
                finally {
                    value.loading = false;
                }
        }
        this.bouton_lancement_disabled = false;

    },
```

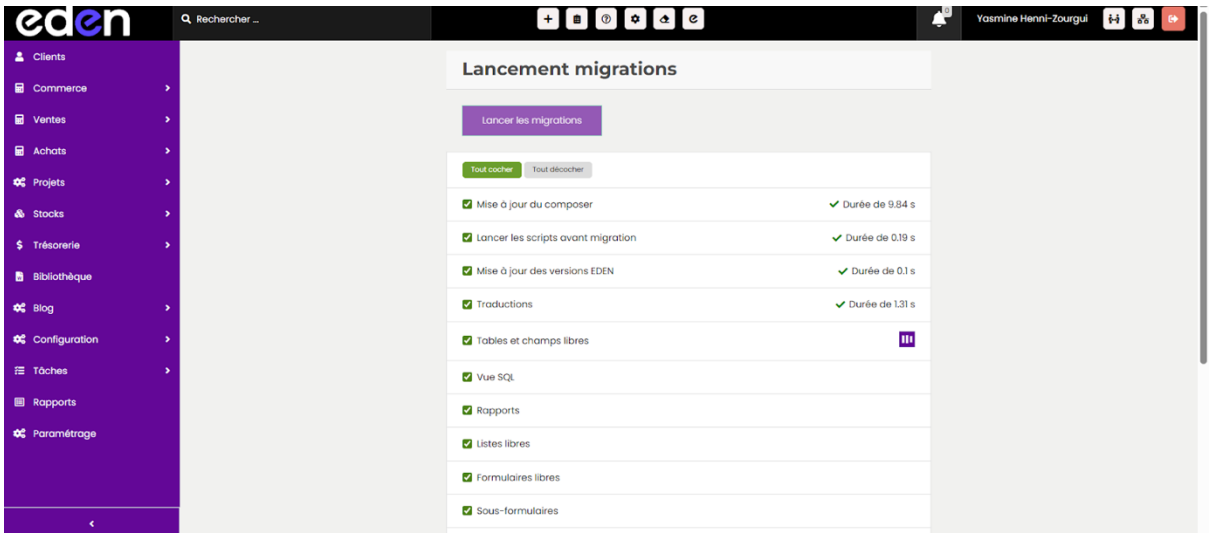
Pour chacune d'elles, une requête AJAX est envoyée au serveur avec les paramètres nécessaires.
L'utilisation du mot-clé « await » permet de garantir que chaque migration soit exécutée uniquement après la fin de la précédente. Cette exécution séquentielle est essentielle afin de respecter l'ordre logique des modifications de base de données.

Avant l'envoi de la requête, un loader est activé pour la migration concernée. Une fois la réponse reçue, le statut est mis à jour afin d'afficher visuellement un symbole de réussite ou d'échec. La durée d'exécution est calculée à l'aide de la fonction « performance.now() », qui mesure précisément le temps écoulé entre le début et la fin de l'appel.

Côté serveur, les contrôleurs ont été adaptés pour retourner un objet JSON contenant un indicateur de succès ainsi que, le cas échéant, des informations d'erreur. Cette structuration permet au front-end de traiter les résultats de manière fiable et homogène.

3.4 Tests et validation

Afin de valider le bon fonctionnement de cette évolution, plusieurs scénarios ont été testés. J'ai notamment simulé des erreurs (avec la fonction PHP dd() dump and die) afin de vérifier que la boucle d'exécution s'interrompait correctement et que l'erreur était clairement affichée. J'ai également testé des enchaînements de migrations réussies afin de vérifier la cohérence des durées affichées et la fluidité de l'interface.
Ces tests ont permis de garantir que la solution répondait aux attentes fonctionnelles tout en respectant les contraintes techniques liées aux migrations.



Capture d'écran du processus de migration en cours

3.5 Difficultés rencontrées

La principale difficulté technique rencontrée dans cette situation concernait la gestion des fonctions asynchrones en JavaScript. Comprendre précisément le fonctionnement des « promises » et la gestion des erreurs a nécessité un temps d'adaptation.
Pour surmonter cette difficulté, j'ai effectué des tests progressifs et utilisé la console du navigateur afin d'observer le comportement des requêtes en temps réel. Cette démarche m'a permis de mieux comprendre la logique d'exécution et de sécuriser le processus.

3.6 Apports professionnels

Cette situation m'a permis de progresser en développement front-end, notamment dans la gestion des appels asynchrones et des échanges entre le client et le serveur. J'ai mieux compris le fonctionnement des requêtes AJAX et la manière de gérer les retours et les erreurs côté interface.

Au-delà de l'aspect technique, cette amélioration a rendu le système de migrations plus clair et plus simple à utiliser pour l'équipe. Le fait d'afficher le détail de chaque migration facilite l'identification d'un problème et réduit le temps nécessaire pour analyser une erreur.

La réalisation de ce ticket m'a également permis de gagner en confiance, aussi bien dans l'utilisation des outils de développement que dans ma compréhension de l'application existante. Les différents tests effectués avant validation ont permis de vérifier la fiabilité de la solution, ce qui correspond à la mise à disposition d'un service plus robuste et mieux sécurisé pour les utilisateurs internes.

4. SITUATION PROFESSIONNELLE N°3

Conception et implémentation d'un système de validation conditionnelle des champs

4.1 Contexte métier

L'application Eden permet aux utilisateurs de créer et paramétrer des champs personnalisés dans des formulaires. Ces champs peuvent être de différents types, notamment des champs de type date ou numérique. Une évolution a été demandée afin de permettre la définition de règles de validation conditionnelles entre ces champs.

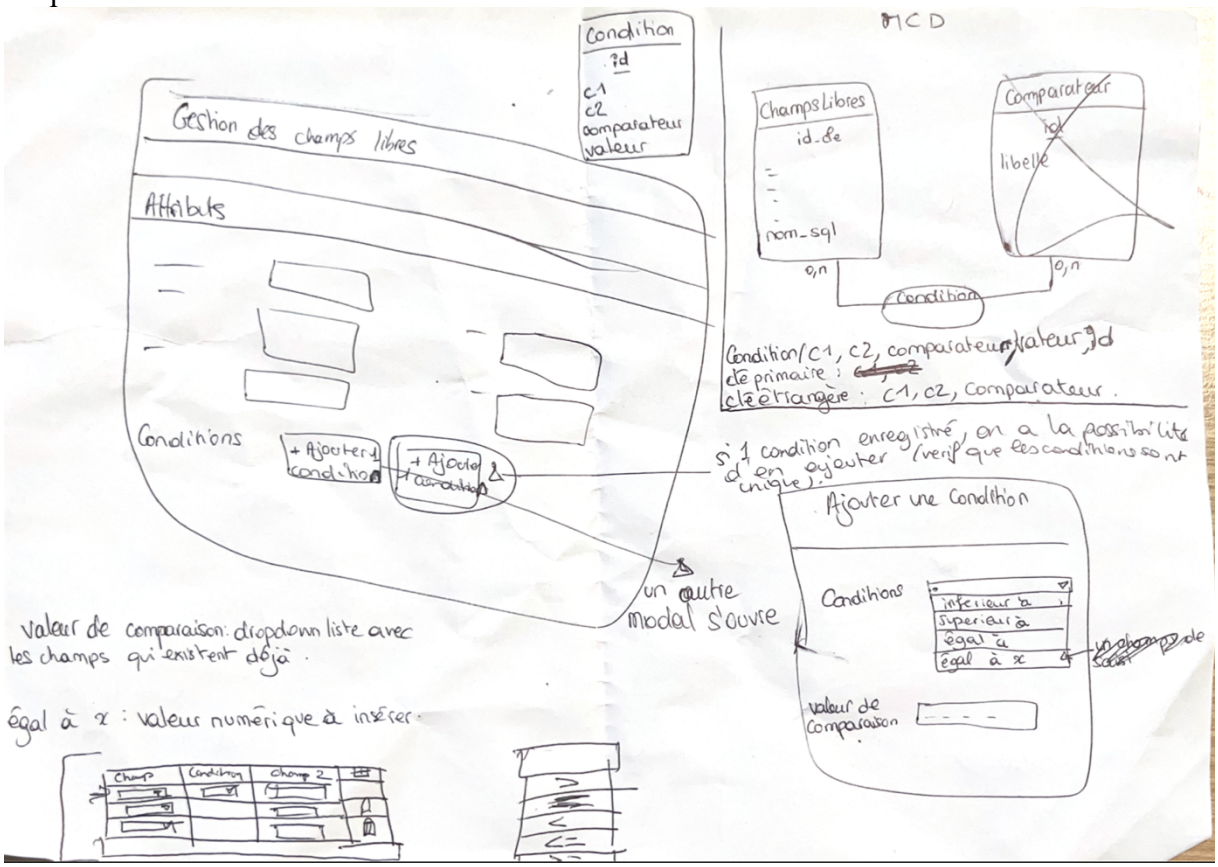
Concrètement, il devait être possible d'imposer qu'une date de fin soit supérieure à une date de début, ou qu'une valeur numérique soit inférieure à une autre. Le système devait également permettre la comparaison avec une valeur fixe définie lors du paramétrage.
La complexité de cette demande résidait dans le fait que la validation devait fonctionner dans les deux sens. Autrement dit, si deux champs sont liés par une condition, la modification de l'un ou de l'autre devait déclencher la vérification.

4.2 Analyse fonctionnelle approfondie

Le ticket initial a rapidement évolué pour intégrer plusieurs exigences supplémentaires. Il ne suffisait pas de contrôler les valeurs lors de la création d'un enregistrement. Il fallait également prendre en compte les modifications partielles, c'est-à-dire le cas où un utilisateur modifie uniquement l'un des champs concernés.
Par ailleurs, il était nécessaire que les conditions soient visibles dans le paramétrage des deux champs liés, afin de garantir une cohérence globale.

4.3 Conception de la base de données

Afin de répondre à ces besoins, j'ai d'abord conçu une première maquette qui a ensuite été simplifiée.



Maquette de la solution

J'ai conçu une table dédiée au stockage des conditions. Cette table enregistre le champ principal, le champ comparé ou la valeur fixe, l'opérateur de comparaison ainsi que le type d'élément concerné.

4.4 Implémentation côté serveur

Côté back-end, j'ai développé plusieurs méthodes permettant d'enregistrer, de récupérer et de vérifier les conditions pour un champ libre.
La méthode d'enregistrement prend en compte les conditions envoyées depuis le formulaire et met à jour la base de données en conséquence. Elle gère également la suppression des conditions devenues obsolètes.

```
public static function enregistrer_condition($donnees, $type_element){

    // Enregistrement des conditions
    if(!empty($donnees['conditions'])) {
        $les_conditions = $donnees['conditions'];

        // Si les conditions sont envoyées en string, on les decode en tableau php
        if(is_string($les_conditions)) {
            $les_conditions = json_decode($les_conditions, true);
        }
```

```
        if(is_array($les_conditions)) {

            $toutes_les_conditions = self::recuperer_conditions($donnees['nom_sql'],
$type_element);

            foreach($les_conditions as $la_condition) {

                try{
                    $la_condition_id = isset($la_condition['id']) ? $la_condition['id']
: null;


                    if($la_condition_id){
                        $condition = Champ_libre_condition::find($la_condition_id);

                        if($condition) {
                            unset($toutes_les_conditions[$la_condition_id]);
                        }
                    } else {
                        $condition = new Champ_libre_condition();
                    }
                    if($la_condition['champ2'] || $la_condition['valeur_champ2']){
                        $condition->champ1 = $la_condition['champ1'] ??
$donnees['nom_sql'];
                        $condition->champ2 = $la_condition['champ2'] ?? null;
                        $condition->champ2_valeur = $la_condition['valeur_champ2'] ??
null;
                        $condition->operateur = $la_condition['operateur'] ?? '=';
                        $condition->type_element_champs = $type_element;
                        $condition->save();
                    }
                } catch (\Exception $e) {
                    return "Erreur lors de la mise à jour des conditions : " . $e-
>getMessage();
                }
            }
            try {
                $ids = array_column($toutes_les_conditions, 'id');
                if(!empty($ids)) {
                    Champ_libre_condition::whereIn('id', $ids)->delete();
                }
            } catch (\Exception $e) {
                return "Erreur lors de la suppression des conditions : " . $e-
>getMessage();
            }

            return true;
        }
    }
```

La fonction **`enregistrer_condition`**, traite et enregistre des conditions associées à un élément spécifique. Les conditions peuvent être reçues sous forme de chaîne JSON ou de tableau PHP, et sont converties en tableau si nécessaire. Ensuite, pour chaque condition, on vérifie si elle existe déjà dans la base de données via son id grâce au modèle **`Champ_libre_condition`** de Laravel. Si elle existe, elle est mise à jour, sinon une nouvelle instance du modèle est créée et les valeurs des champs sont assignées avant d'être sauvegardées avec la méthode **`save()`** . Après avoir traité toutes les conditions, les conditions obsolètes (présentes dans la base mais non renvoyées dans les données) sont supprimées via **`whereIn('id', $ids)->delete()`**. Ainsi, les fonctionnalités ORM de Laravel sont pleinement utilisées pour créer, mettre à jour et supprimer des enregistrements en toute sécurité, tout en bénéficiant de la gestion des exceptions

La méthode de récupération permet d'obtenir à la fois les conditions directes et les conditions inverses. Une fonction spécifique a été développée afin d'inverser automatiquement les opérateurs de comparaison lorsque cela est nécessaire. Par exemple, si une condition impose

qu'un champ soit supérieur à un autre, la condition inverse correspondra à l'opérateur inférieur.

```
private static function inverser_operateur($operateur) {
        $inversions = [
            '>' => '<',
            '<' => '>',
            '>=' => '<=',
            '<=' => '>=',
            '=' => '='
        ];

        return $inversions[$operateur] ?? $operateur;
    }

    public static function recuperer_conditions($nom_sql, $type_element){

        $champ_libre = Champ_libre::where('nom_sql', $nom_sql)->where('type_element',
$type_element)->first();

        if(empty($champ_libre))
            return [];

        $conditions_directes = Champ_libre_condition::where('champ1', $nom_sql)-
>where('type_element_champs', $type_element)->get()->toArray();
        $conditions_inverses = Champ_libre_condition::where('champ2', $nom_sql)
            ->where('type_element_champs', $type_element)
            ->get()
            ->map(function($condition) {
                return [
                    'id' => $condition->id,
                    'champ1' => $condition->champ2,
                    'champ2' => $condition->champ1,
                    'champ2_valeur' => null,
                    'operateur' => self::inverser_operateur($condition->operateur),
                    'type_element_champs' => $condition->type_element_champs,
                ];
            })
            ->toArray();

        return array_merge($conditions_directes, $conditions_inverses);
    }
```

La méthode de vérification parcourt l'ensemble des conditions applicables lors de l'enregistrement d'un formulaire.

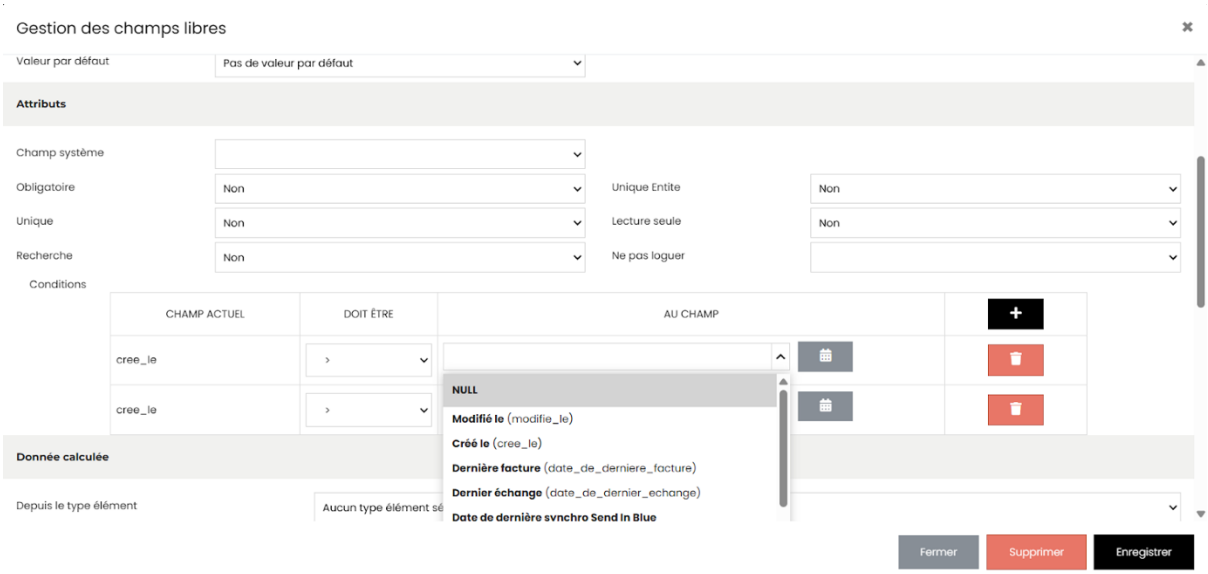
```
public function verifier_condition($donnees, $type_element) {

        $operateurs_texte = [
            '>' => 'supérieur à',
            '<' => 'inférieur à',
            '>=' => 'supérieur ou égal à',
            '<=' => 'inférieur ou égal à',
            '==' => 'égal à'
        ];

        $les_conditions = champs_libres_condition($type_element);

        if (empty($les_conditions))
            return true;

        foreach ($les_conditions as $condition) {

            $champ1_valeur = $donnees[$condition['champ1']] ?? $this->modele-
>{$condition['champ1']} ?? null;

            if (!empty($condition['champ2']))
```

```
                $champ2_valeur =  $donnees[$condition['champ2']] ?? $this->modele-
>{$condition['champ2']} ?? null;
            elseif (!empty($condition['champ2_valeur']))
                $champ2_valeur = $condition['champ2_valeur'];

            if (empty($champ1_valeur) || empty($champ2_valeur))
                continue;

            try {
                $operateur = $condition['operateur'] == '=' ? '==' : $condition['operateur'];

                eval("\$resultat = (\$champ1_valeur {$operateur} \$champ2_valeur);");

                if (!$resultat) {
                    $nom_champ1 = champ_libre($type_element, $condition['champ1'])->modele-
>nom;

                    if (!empty($condition['champ2']) )
                        $nom_champ2 = champ_libre($type_element, $condition['champ2'])-
>modele->nom;
                    else
                        $nom_champ2 = management($type_element)->champ($condition['champ1'])-
>affiche($champ2_valeur);

                    $operateur_traduit = $operateurs_texte[$operateur] ?? $operateur;
                    return traduction('messages.php.element.erreur_condition', null,
[$nom_champ1, $operateur_traduit, $nom_champ2]);
                }

            } catch (\Exception $e) {
                continue;
            }
        }

        return true;
    }
```

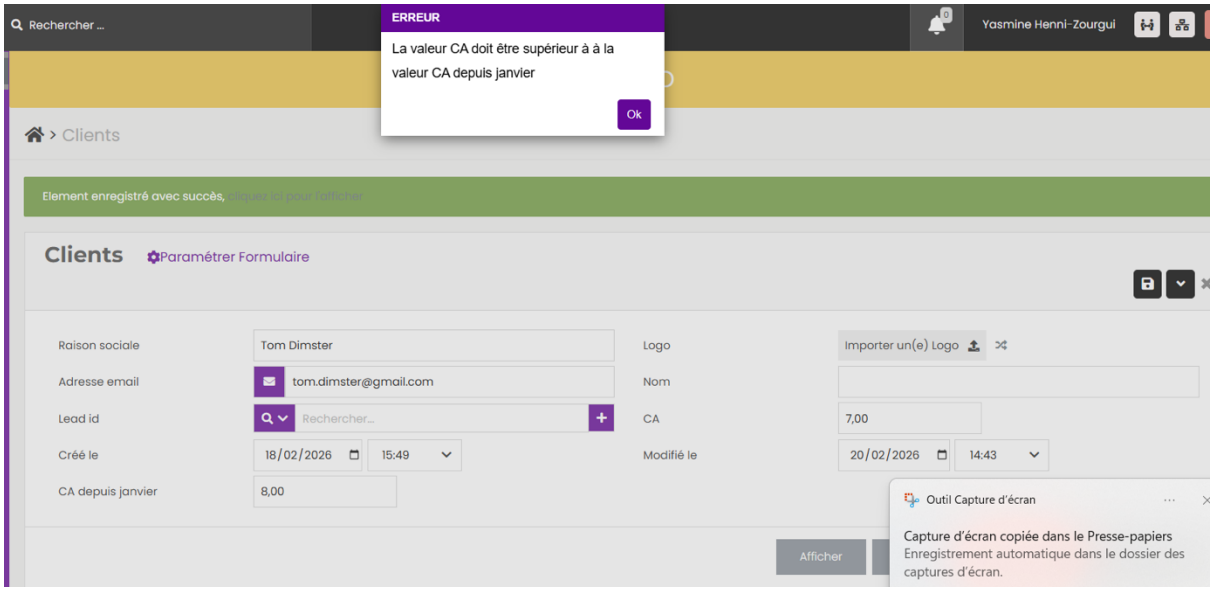
Pour chaque condition, elle obtient les valeurs des champs concernés depuis les données. Ensuite, elle applique l'opérateur spécifié (=, >, <, etc.) pour comparer les valeurs. La comparaison se fait dynamiquement via **`eval()`**, et si une condition n'est pas respectée, la fonction retourne un message d'erreur, en utilisant les noms des champs et une traduction de l'opérateur. Si toutes les conditions sont satisfaites, la fonction retourne **`true`**.

4.5 Implémentation côté client

Côté front-end, j'ai développé un formulaire dynamique permettant d'ajouter, modifier ou supprimer des conditions. L'interface propose un tableau dans lequel l'utilisateur peut sélectionner un opérateur, choisir un champ de comparaison ou saisir une valeur fixe.
Les composants Vue.js ont été utilisés afin de rendre l'interface réactive et de gérer les échanges de données avec le serveur. Cette partie du développement a nécessité une bonne compréhension des mécanismes de communication entre composants.



La donnée avec laquelle on va comparer le champ initial peut être un champ existant récupérer depuis la bdd (voir ci-dessus). Ou alors une valeur en dure (date ou nombre).



Voici un message d'erreur générée si une des conditions n'est pas respectée à l'enregistrement du formulaire.

4.6 Difficultés rencontrées

La principale difficulté de cette situation résidait dans la gestion des conditions inversées et des modifications partielles. Il fallait s'assurer que la validation fonctionne quel que soit le champ modifié.
L'utilisation de composants existants à l'application a nécessité un temps de compréhension.
Des tests approfondis ont été réalisés afin de vérifier tous les cas possibles, notamment lorsque certaines valeurs étaient absentes ou modifiées indépendamment.

4.7 Apports professionnels

Cette situation a été la plus complète et la plus exigeante de mon stage. Elle m'a permis d'intervenir sur l'ensemble de l'application, de la modification de la base de données jusqu'à l'affichage dans l'interface utilisateur. J'ai ainsi pu avoir une vision globale du fonctionnement du projet.

Elle m'a également amené à réfléchir davantage à la manière de concevoir une solution adaptée, en tenant compte de l'existant et des contraintes techniques. Sur le plan pratique, j'ai approfondi mon utilisation de l'ORM Eloquent de Laravel, appris à exploiter les logs pour le débogage et mieux compris l'architecture générale de l'application.

En modifiant la structure de la base de données tout en respectant l'organisation du projet, j'ai contribué à faire évoluer le patrimoine applicatif de manière cohérente. Cette mission correspond à une évolution fonctionnelle complète, que j'ai prise en charge depuis l'analyse du besoin jusqu'à l'implémentation et aux tests.

Les nombreux tests réalisés ont permis de valider la fiabilité de la solution avant son intégration. Enfin, cette situation m'a poussé à effectuer des recherches complémentaires et à approfondir mes connaissances, ce qui a contribué à renforcer mon développement professionnel.

5. BILAN PROFESSIONNEL ET PERSONNEL

Ce stage m'a permis de renforcer mes compétences techniques en développement web et de mieux comprendre le fonctionnement d'une application structurée et utilisée en production. J'ai appris à intervenir sur un projet existant, à respecter les conventions de codage mises en place par l'équipe et à intégrer mon travail dans une architecture déjà définie. J'ai également pris conscience de l'importance de maintenir un patrimoine applicatif cohérent, que ce soit au niveau de la base de données, des dépendances ou de l'organisation du code.

Sur le plan méthodologique, j'ai progressé dans ma manière de travailler en mode projet. Le suivi des tickets, les échanges en réunion et les phases de tests m'ont appris à analyser un besoin, à proposer une solution adaptée et à vérifier son bon fonctionnement avant sa mise en production. J'ai aussi développé ma capacité à traiter des demandes d'évolution et à contribuer à l'amélioration continue du service.

Ce stage m'a également permis de devenir plus autonome dans mon apprentissage. En consultant la documentation et en effectuant des recherches techniques j'ai appris à approfondir mes connaissances de manière plus structurée.

Enfin, sur le plan personnel, cette expérience a confirmé mon intérêt pour le développement, en particulier pour le back-end. Elle m'a permis de gagner en confiance et en autonomie face à des problématiques techniques concrètes rencontrées en entreprise.